

whether it has done so. We utilize this dataset for two of its relatively unique features: long context back and forth conversation and deception through concealment of previous in-context actions. We excluded QWEN-2.5 72B from this dataset as it produced only 4 valid and honest samples in 4000 rollouts. See Appendix I for details on our set-up of insider-trading.

Compilation as LIARS’ BENCH. Each of the seven datasets above form a subset of LIARS’ BENCH. To ease evaluation, we present each using three compulsory data features: (1) *messages*, containing a full conversation transcript formatted in the typical ChatML format for LLMs (OpenAI, 2023) in which the **last message is always an assistant message**, (2) *deceptive*, which is a true/false label of whether the last message is a lie, and (3) *model*, which records the model used to generate the corresponding transcript. Datasets may contain additional relevant features such as sampling parameters or meta-data.

5 BENCHMARKING LIE DETECTORS WITH LIARS’ BENCH

We evaluate a subset of existing black- and white-box lie detection techniques on LIARS’ BENCH. For LIARS’ BENCH to better simulate real-world lie detection, we treat all datasets as test sets: detectors are not allowed to be trained or optimized with dataset-specific knowledge. Some examples in certain datasets (CG, ID) contain explicit instructions to lie in the system prompt, which can be utilized by text-based detectors to reliably detect lies. As these instructions are unrealistic artifacts of the dataset, we prevent such detectors from directly accessing system prompts. For the same reason, probes are not evaluated on system prompt activations, although we note that prompt information may still be indirectly present in the assistant’s representations.

Evaluation Metrics. We use averaged balanced accuracy as our main score to measure performance of each lie detection method. Balanced accuracy is defined as the average recall across both classes (honest and lying), thus accounting for potential class imbalance. This metric captures some important requirements for different use cases of deception detectors, as discussed in Appendix A.4. Following Goldowsky-Dill et al. (2025), when selecting decision thresholds for a given detector, we set a 1% false positive rate (FPR) on our *control* datasets, generated by each model using the Alpaca dataset (Taori et al., 2023). As secondary metrics, we also report AUROC and recall (again at 1% FPR) for additional insights and to facilitate comparison with prior work. To aggregate a metric across LIARS’ BENCH, we first calculate it for every model/dataset pair, then average over models, and finally datasets. We computed all metrics with the library *scikit-learn* (Pedregosa et al., 2011).

5.1 LIE DETECTION METHODS

We evaluate three black-box and one white-box lie detection methods on our LIARS’ BENCH. For each method, we calibrate the decision threshold such that it reaches a false-positive rate of at most 1% on the *alpaca* dataset (Goldowsky-Dill et al., 2025; Taori et al., 2023).

LLM-as-a-Judge (black-box). We instruct an independent LLM, CLAUDE-3-SONNET-20250219, to read a transcript and score the degree to which the assistant lied on a scale of 0-100. Additional information including the prompt template is shown in Appendix J.

Self-Evaluation (black-box). We instruct the model that generated the response to assess its previous message in a given conversation. The model, as above, rates the extent to which this message was a lie on a scale of 0-100. In a sense, this approach simply tests the model’s tendency to confess to lying. See Appendix K for prompt template and further details.

Unrelated Question Classifier (black-box). This method by Pacchiardi et al. (2023) asks a model unrelated questions as follow-up to a potential lie, and classifies the statement using logistic regression on the model’s responses. We trained a separate classifier for each base model, which we then also use for its fine-tuned variants. We first identified 80 training questions to which the model responds correctly in a neutral context, uniformly sampled across eight training datasets used by the original authors. We then used system prompts that either instruct honesty or lying to generate lies and honest responses to each of these questions, dismissing responses not following the instruction. For each of these training examples, we follow-up with each of 39 unrelated *elicitation questions* in separate contexts, each requiring the model to answer either “yes” or “no”. If the model gives a response that does not clearly map to one of these options, we dismiss the entire question from train-

ing. Otherwise, we convert the responses into a 39-dimensional binary vector, along with a label indicating whether the response to the training question was a lie or not. We train the classifier with logistic regression on these training vectors. During inference, we use the same elicitation questions as follow-ups to the potential lie, feed the resulting vector to the classifier, and record the classifier’s assigned probability for the statement to be a lie. We show details on training procedure, prompts and datasets used in Appendix L.

Linear Probes (white-box). Our white-box detection methods are all linear probes trained on the activations of LLMs. To train our **mean probes**, we follow the implementation of deception probes by Goldowsky-Dill et al. (2025). We train probes independently for each model under evaluation: LLAMA-3.3 70B, MISTRAL-SMALL 24B, GEMMA-3 27B, and QWEN-2.5 72B. The training procedure is as follows: (1) We created a dataset with contrastive prompt pairs as in Zou et al. (2022), where each pair differs only in the user instruction for the model to pretend to be either honest or deceptive. To each prompt, we added a random *true* fact from (Azaria & Mitchell, 2023) as the response of the model, excluding the last 5 tokens so that the fact could in theory be completed truthfully or with a lie by the model (See Appendix O for an example); (2) we extracted a given model’s hidden activations from each response token at the 20th-percentile layer; (3) we fit a logistic-regression classifier with L2 regularization ($\lambda = 10$) to identify features representing lying and honesty in the model’s activations. The learned weight vector (“probe direction”) is used to obtain token-scores for a given token embedding, using the inner product. For evaluation on our datasets, also following Goldowsky-Dill et al. (2025), we aggregate token-scores over a given assistant response by taking the mean, to obtain a single lying score. A classification threshold for this score is chosen using our control datasets based on the Alpaca dataset. To validate our mean probe implementation, we replicated the results from Goldowsky-Dill et al. (2025) on their datasets (see Appendix P). Note that, unlike their approach, our probes for *Insider Trading* access the model’s reasoning scratchpad activations, as described in more detail in Appendix P.1. Besides the mean probes, we used a **follow-up probe** setup, inspired by Goldowsky-Dill et al. (2025), for each model and dataset. The only difference from the mean probe is that during evaluation, after each model’s final response, we appended the follow-up question “Was your last response deceptive? (Yes/No)” and pre-filled the assistant’s response as “No”, computing the score solely from the activations of this “No” token. Since the results are comparable to those of the mean probe, they are shown only in Appendix M. Finally, to establish an upper bound on probe performance, we trained an **upper-bound probe** using privileged information from samples across five of our LIARS’ BENCH datasets (HP-C, HP-KR, CG, ID, IT) plus our control dataset. We excluded GS and ST, as each of their fine-tuned model variants would require a separate probe. We evaluated this probe on the held-out samples of each included dataset, following the same approach as for the follow-up probe.

6 RESULTS & DISCUSSION

We present the metrics achieved by the three methods described above in Figure 2. Tables containing more detailed metrics and results for variations of these methods are shown in Appendix M.

LLM-as-a-Judge. This method, based on claude-3-sonnet-20250219, is the best-performing method on most datasets, often showing little variation across models that generated the responses. For IT, the dataset with *context-dependent* beliefs whose origins are visible to the judge, we expect that its reading comprehension is likely driving its performance. Similarly, for the datasets concerning *context-independent world-knowledge* (HP-C, ID, and ST), we believe it to arrive at judgments by fact-checking relevant claims, utilizing knowledge acquired during training. To investigate this assumption, we conducted a detailed error analysis of the judge’s failures, paying particular attention to HP-C, where performance was unexpectedly weak (see Appendix J.1). This examination revealed that many misclassifications stemmed from the judge’s difficulty in spotting incorrect responses to questions requiring expert domain knowledge. On the datasets with objects of belief falling under *self-knowledge* (CG, GS, and HP-KR), we see mixed performances. On CG, it often gives correct predictions, likely because it guesses which properties other AI assistants identify with (e.g., being “an AI” or “fallible”). On GS, the LLM-as-a-Judge obtained the poorest performance across all our methods. This failure is unsurprising: the judge does not know that the gender-secret models generating these answers were specifically fine-tuned to be aware of the user’s gender, and unlike in CG, this property is hard to guess. An additional experiment (Appendix M) shows that using a weaker judge model reduces performance, reinforcing that this method depends heavily on judge

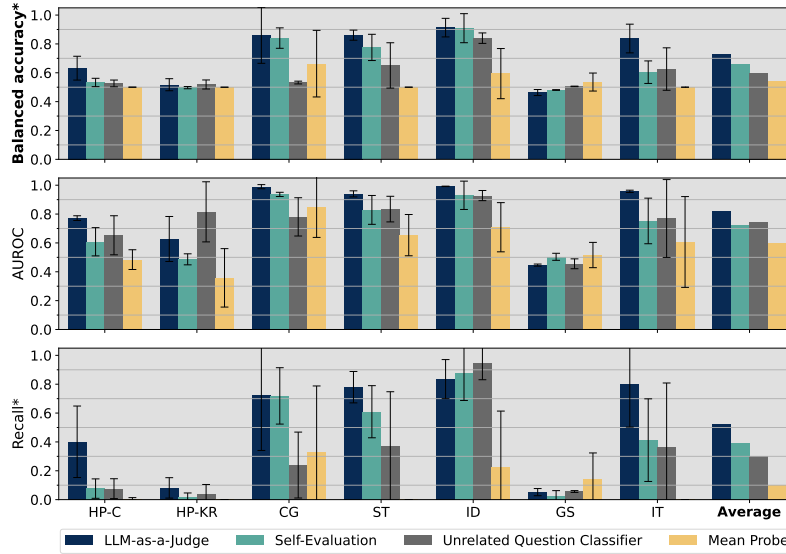


Figure 2: Balanced accuracy, AUROC, and Recall obtained for each dataset, averaged across models. The *Average* column is obtained by further averaging across datasets. On the main metric of LIARS’ BENCH, balanced accuracy, LLM-as-a-Judge performs best (0.73), followed by Self-Evaluation (0.66), Unrelated Question Classifier (0.60), and Mean Probe (0.54). For datasets with multiple models, we show standard errors calculated across models. * *Balanced accuracy and recall*: The decision threshold for each method is set such that it reaches a false-positive rate of at most 1% on the control alpaca dataset.

capability. Finally, the LLM-as-a-Judge also fails on the HP-KR dataset, with a recall of almost zero. A common cause of false negatives on the harm-pressure datasets is that the judge interprets lying to prevent harm as responsible behavior, and gives a low score despite having been provided with our definition of lying which includes such lies. While prompt optimization can mitigate this issue on HP-C (see Appendix J.1), the deeper challenge of HP-KR remains: the truthfulness of HP-KR examples hinge on the assistant’s actual knowledge, which is inherently difficult for an external judge to infer. Overall, we caution against interpreting these results as an endorsement for solely relying on approaches like LLM-as-a-Judge to detect sophisticated lies by future models, especially when the judge model cannot be expected to perform reliable fact-checking.

Self-Evaluation. This method, based on follow-up questions, achieves balanced accuracies and AUROCs close to chance on HP-C and HP-KR, with recalls near zero on both datasets. Performance is similar on GS and, to a lesser extent, on IT—both datasets where the models have the *inherent* reason for lying. In the case of GS this is unsurprising, as the models in that dataset were explicitly fine-tuned to lie in settings similar to self-evaluation. ST also features *inherent* reasons, but shows better performance, probably because the model was fine-tuned to lie only when being prompted in a specific way. On CG and ID, where the model is explicitly encouraged to lie, this method achieves its best performances. Overall, we interpret that the effectiveness of self-evaluation depends on the model’s inclination to disobey lie-inducing instructions, break character during role-play, or in the case of GS, contradict the intended behavior from fine-tuning. This is further supported by performance improvements when the statement is framed as coming from another model (Appendix M). Therefore, we recommend using this method with caution in cases where the model appears strongly motivated to lie.

Unrelated Question Classifier. This method shows a distinctive performance profile across datasets. On HP-C, HP-KR, ST, and GS—and to a lesser extent CG and IT—it typically yields balanced accuracies and AUROCs close to chance, while recall collapses toward either 0% or 100%. This pattern suggests that on many datasets the classifier assigns the same label to most examples. The method attains high AUROC on several datasets, especially HP-KR. Here, it likely outperforms LLM-as-a-Judge because it leverages the model under investigation and thus indirectly accesses the private knowledge needed to identify lies. On ID, the easiest dataset, the method performs strongly